

1
Week

3
Day

5
Topics

Prototypes & OOP
Focus

Topic 1 — Prototype Chain & Object.create()

JavaScript ek **prototype-based** language hai — classical OOP (Java/C++) jaisi nahi. Har object ke andar ek hidden property hoti hai: **[[Prototype]]** (accessible via `__proto__` ya `Object.getPrototypeOf()`). Jab koi property ya method kisi object mein nahi milta, JS automatically uske prototype mein dhundta hai — yahi **Prototype Chain** hai.

[[Prototype]]

Har object ka hidden link — uske parent object ki taraf point karta hai.

Prototype Chain

Property lookup chain —
object → prototype →
prototype →
Object.prototype → null.

Object.create()

Naya object banao with explicit prototype — clean inheritance without constructors.

hasOwnProperty

Check karo ki property object ki khud ki hai ya inherited — important for loops.

Object.create() — Prototype Chain

```
// Every object has a prototype
const animal = {
  breathe() { return `${this.name} is breathing`; },
  eat() { return `${this.name} is eating`; },
};

// Object.create() – set prototype explicitly
const dog = Object.create(animal);
dog.name = 'Bruno';
dog.bark = function() { return `${this.name} says: Woof!`; };
dog.breathe(); // 'Bruno is breathing' – inherited from animal
dog.bark(); // 'Bruno says: Woof!' – own method

// Prototype chain verification
Object.getPrototypeOf(dog) === animal; // true
dog.hasOwnProperty('name'); // true – own property
dog.hasOwnProperty('breathe');// false – inherited!

// Multi-level chain
const puppy = Object.create(dog);
puppy.name = 'Max';
puppy.breathe(); // 'Max is breathing' – chain: puppy→dog→animal
puppy.bark(); // 'Max says: Woof!' – chain: puppy→dog
```

Chain Visualization & instanceof

```
// Visualizing the chain
// puppy.__proto__ === dog
```

```
// dog.__proto__ === animal
// animal.__proto__ === Object.prototype
// Object.prototype.__proto__ === null <-- end of chain
// Property lookup order:
// puppy.breathe() -->
// 1. puppy.breathe? NO
// 2. dog.breathe? NO
// 3. animal.breathe? YES -> Execute!
// Object.create(null) – no prototype chain (pure dict)
const dict = Object.create(null);
dict.key = 'value';
// dict.toString – undefined (no Object.prototype methods!)
// Useful for: lookup tables, JSON-like stores
// Checking full chain
dog instanceof Object; // true – Object is in chain
'breathe' in dog; // true – checks entire chain
dog.hasOwnProperty('breathe'); // false – not OWN property
```

■ Critical: for...in vs Object.keys() — Inherited Properties

for...in loop inherited properties bhi iterate karta hai — unexpected bugs ho sakte hain!

Object.keys() sirf OWN enumerable properties deta hai — safer for iteration.

Hamesha hasOwnProperty check karo agar for...in use kar rahe ho purane code mein.

Example: for(let k in dog) → 'name','bark','breathe','eat' (inherited bhi!)

Object.keys(dog) → ['name','bark'] (sirf own properties)

Topic 2 — ES6 Class Syntax, Constructor & Methods

ES6 **class** keyword ek **syntactic sugar** hai prototype system ke upar. Under the hood sab kuch wahi prototype chain hai — bas writing style cleaner hai. **constructor()** method object create hone par automatically call hota hai. Methods class ke prototype pe define hote hain — har instance mein copy nahi hoti.

Constructor Function (ES5)

```
// Old way – Constructor Function
function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.greet = function() {
  return `Hi, I'm ${this.name}`;
};
Person.prototype.isAdult = function() {
  return this.age >= 18;
};
const p = new Person('Rahul', 20);
p.greet(); // 'Hi, I'm Rahul'
p.isAdult(); // true
```

class Syntax (ES6)

```
// ES6 Class – same result, cleaner
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    return `Hi, I'm ${this.name}`;
  }
  isAdult() {
    return this.age >= 18;
  }
}
const p = new Person('Rahul', 20);
p.greet(); // 'Hi, I'm Rahul'
p.isAdult(); // true
// Under the hood: SAME prototype chain!
```

Complete Class with Private Fields, Getters & Static Methods

```
// Full-featured class with all member types
class BankAccount {
  // Class field (modern syntax)
  #balance = 0; // Private field – # prefix
  #transactions = [];
  constructor(owner, initialDeposit = 0) {
    this.owner = owner;
    this.accountNo = Math.random().toString(36).slice(2, 10).toUpperCase();
    if (initialDeposit > 0) this.deposit(initialDeposit);
  }
  // Instance methods
  deposit(amount) {
    if (amount <= 0) throw new Error('Amount must be positive');
    this.#balance += amount;
    this.#transactions.push({ type: 'credit', amount, date: new Date() });
    return this; // method chaining!
  }
  withdraw(amount) {
    if (amount > this.#balance) throw new Error('Insufficient funds');
    this.#balance -= amount;
    this.#transactions.push({ type: 'debit', amount, date: new Date() });
    return this;
  }
  // Getters
  get balance() { return this.#balance; }
  get history() { return [...this.#transactions]; }
  get summary() {
```

```

return `${this.owner} [${this.accountNo}]: Rs.${this.#balance}`;
}

// Static method
static validateAmount(amount) {
return typeof amount === 'number' && amount > 0;
}
}

const acc = new BankAccount('Priya', 5000);
acc.deposit(2000).withdraw(500); // method chaining
acc.balance; // 6500
acc.summary; // 'Priya [X2K9M1]: Rs.6500'
// acc.#balance – SyntaxError! private field
BankAccount.validateAmount(500); // true (static – no instance needed)

```

■ Critical: class is NOT hoisted like function

function Person(){} ko before declaration call kar sakte ho (hoisted).

class Person{} ko PEHLE define karna padta hai — temporal dead zone mein hai.

class ke andar 'use strict' automatically ON hai — sloppy mode code kaam nahi karega.

Methods class ke prototype pe hain — arrow functions as methods class field use karo tab:

class C { method = () => this.x; } // arrow — 'this' fixed

class C { method() { return this.x; } } // regular — 'this' depends on caller

Topic 3 — Inheritance with extends & super

extends keyword se ek class doosri class se inherit kar sakti hai — is relationship ko **IS-A** kehte hain (Dog IS-A Animal). **super()** parent class ke constructor ko call karta hai — child class constructor mein **this** use karne se pehle super() call MUST hai.

Multi-level Inheritance — Animal → Dog → GuideDog

```

// Base class (Parent)
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
    this.alive = true;
  }
  speak() { return `${this.name} says: ${this.sound}`; }
  eat(food) { return `${this.name} eats ${food}`; }
  toString() { return `[Animal: ${this.name}]`; }
}

// Child class – inherits Animal
class Dog extends Animal {
  constructor(name, breed) {
    super(name, 'Woof'); // MUST call super() FIRST before using 'this'
    this.breed = breed; // own property
  }
}

```

```

fetch(item) { return `${this.name} fetches the ${item}!`; }
// Override parent method
toString() {
return `[Dog: ${this.name} (${this.breed})]`;
}
}

class GuideDog extends Dog {
constructor(name, breed, owner) {
super(name, breed); // calls Dog's constructor
this.owner = owner;
}
guide() { return `${this.name} guides ${this.owner} safely.`; }
}

const rex = new Dog('Rex', 'Labrador');
const buddy = new GuideDog('Buddy', 'Golden Retriever', 'Rahul');
rex.speak(); // 'Rex says: Woof' – inherited
rex.eat('bone'); // 'Rex eats bone' – inherited
rex.fetch('ball');// 'Rex fetches the ball!' – own
rex.toString(); // '[Dog: Rex (Labrador)]' – overridden
buddy.guide(); // 'Buddy guides Rahul safely.'
buddy.speak(); // 'Buddy says: Woof' – chain: GuideDog→Dog→Animal

// instanceof checks
buddy instanceof GuideDog; // true
buddy instanceof Dog; // true
buddy instanceof Animal; // true
buddy instanceof Object; // true – everything is Object

```

super.method() — Extending Parent Behavior

```

// super in methods – call parent version
class Vehicle {
constructor(make, model, year) {
this.make = make;
this.model = model;
this.year = year;
}
describe() {
return `${this.year} ${this.make} ${this.model}`;
}
startEngine() { return 'Vroom! Engine started.'; }
}

class ElectricVehicle extends Vehicle {
constructor(make, model, year, range) {
super(make, model, year);
this.range = range;
this.battery = 100;
}
// super.method() – extend parent behavior, not replace
describe() {

```

```
return `${super.describe()} | Electric | Range: ${this.range}km`;
}

startEngine() {
return 'Silent start... Motors spinning.';
}

charge(pct) {
this.battery = Math.min(100, this.battery + pct);
return `Battery: ${this.battery}%`;
}
}

const tesla = new ElectricVehicle('Tesla', 'Model 3', 2024, 500);
tesla.describe(); // '2024 Tesla Model 3 | Electric | Range: 500km'
tesla.startEngine(); // 'Silent start... Motors spinning.'
tesla.charge(20); // 'Battery: 100%' (already 100)
```

■ Critical: super() Rules — Must Know

Rule 1: Child constructor mein this se pehle super() ZAROORI hai. Bhule to: ReferenceError.

Rule 2: `super()` sirf constructor mein hota hai, methods mein `super.methodName()` hota hai.

Rule 3: extends ke baad super() call na karna = 'Must call super constructor in derived class'.

Rule 4: Agar child class mein constructor nahi likha, automatically parent ka use hota hai.

Topic 4 — Static Methods, Getters & Setters

Static methods/properties class ke instances pe nahi, **class khud pe** call hote hain. Utility functions, factory methods ke liye useful. **Getters** property ki tarah access hote hain (no () needed), **setters** value assign karne par run hote hain — validation add karne ke liye perfect.

Getters, Setters & Static Methods — Temperature Class

[illegible]

[illegible]

■ Static Methods Real-World Uses

Factory methods: `User.fromJSON(data)`, `Date.now()`, `Promise.all()`

Utility/helper: `Array.isArray()`, `Number.isInteger()`, `Math.max()`

Config/constants: static MAX SIZE = 100; — class level constant

Singleton pattern: class AppConfig { static instance = null; static getInstance() {...} }

Topic 5 — Mixins Pattern

JavaScript mein ek class sirf ek parent se inherit kar sakti hai (**single inheritance**). Lekin kabhi kabhi multiple sources se behavior chahiye — yahan **Mixins** kaam aate hain. Mixin ek function hai jo ek class leta hai, usme methods add karta hai aur return karta hai. Ye **composition over inheritance** principle follow karta hai.

Mixins — Serializable + Validatable + Timestamps

```
// Mixin functions – reusable behavior blocks
const Serializable = (Base) => class extends Base {
  toJSON() { return JSON.stringify(this); }
  toString(){ return `[${this.constructor.name}] ${this.toJSON()}`; }
  clone() { return Object.assign(Object.create(Object.getPrototypeOf(this)), this); }
};

const Validatable = (Base) => class extends Base {
  validate() {
    const rules = this.constructor.validationRules || {};
    const errors = [];
    for (const [field, rule] of Object.entries(rules)) {
```

```

if (rule.required && !this[field])
errors.push(`${field} is required`);
if (rule.minLength && this[field]?.length < rule.minLength)
errors.push(`${field} min length: ${rule.minLength}`);
}
return { valid: errors.length === 0, errors };
}
};

const Timestamps = (Base) => class extends Base {
  constructor(...args) {
    super(...args);
    this.createdAt = new Date().toISOString();
    this.updatedAt = new Date().toISOString();
  }
  touch() { this.updatedAt = new Date().toISOString(); return this; }
};

// Base class
class Model {
  constructor(data) { Object.assign(this, data); }
}

// Mix multiple behaviors into User
class User extends Serializable(Validatable(Timestamps(Model))) {
  static validationRules = {
    name: { required: true, minLength: 2 },
    email: { required: true },
  };
}

const user = new User({ name: 'Rahul', email: 'rahul@example.com', role: 'admin' });
user.validate(); // { valid: true, errors: [] }
user.toJSON(); // JSON string
user.createdAt; // ISO timestamp
user.touch(); // updates updatedAt
const badUser = new User({ name: 'R' });
badUser.validate();
// { valid: false, errors: ['email is required', 'name min length: 2'] }

```

■ Composition vs Inheritance — Kab kya use karein?

Inheritance (extends): IS-A relationship — Dog IS-A Animal. Tight coupling, deep hierarchy avoid karo.

Mixin/Composition: HAS-A relationship — User HAS Timestamps, HAS Validation. Flexible, reusable.

Rule of thumb: 2 levels se zyada inheritance hierarchy ka matlab design revisit karo.

React mein bhi: 'favor composition over inheritance' — Hooks isi pattern pe based hain.

Day 3 Learning Outcomes

Prototype chain kaise kaam karta hai — visualize karke explain kar paoge.

ES6 class syntax mein constructor, methods, getters, setters, static sab likhna aayega.

Inheritance extends + super properly use kar paoge — ReferenceError avoid.

Private fields (#), method chaining, factory patterns implement kar paoge.

Mixins se multiple behaviors compose kar paoge without deep inheritance.